Eco-Adaptive Home Storage: Multi-Objective Reinforcement Learning for Smart Grid Energy Management

Ilyas Madah, Adam Younsi, and Zaid El Kasemy

Reinforcement Learning Group Project
{adam.younsi, ilyas.madah, zaid.elkasemy}@student-cs.fr

Abstract. We present a comparative study of reinforcement learning approaches for intelligent home battery management in smart grid environments. Our system optimizes a multi-objective reward function balancing electricity costs, carbon emissions, and battery wear. We implement and evaluate two learned agents : Deep Q-Network (DQN) with neural network function approximation and SARSA with tile coding against multiple rule-based baselines. Through systematic hyperparameter ablation and statistical evaluation with 95% confidence intervals, we demonstrate that SARSA with optimized tile coding achieves the best performance, with mean daily savings of 2.19 and 3.34 kg CO_2 avoided, outperforming both DQN and expert-designed heuristics. Our results highlight that linear function approximation with appropriate feature engineering can compete with deep learning approaches in structured domains. . Link to the code with a comprehensive notebook highlighting the different results : `https://github.com/ilyass7m/Smart_Grid_RL/tree/master`

Keywords: Reinforcement Learning · Smart Grid · Battery Management · DQN · SARSA · Tile Coding · Multi-Objective Optimization

1 Introduction

The integration of renewable energy and residential battery storage offers significant potential for reducing electricity costs and carbon footprints. However, achieving optimal energy management is complicated by the stochastic interactions between solar generation, household demand, fluctuating grid prices, and battery degradation constraints. Reinforcement learning (RL) provides a robust framework for this sequential decision-making challenge, enabling agents to derive optimal control policies through environmental interaction without requiring explicit predictive models of consumption or price dynamics. In this study, we formulate the home battery management problem as a Markov Decision Process (MDP) and evaluate the performance of several RL architectures:

- **DQN**: Deep Q-Network with experience replay and target networks [1]
- **SARSA**: On-policy temporal difference control with tile coding [2]

– **Rule-based baselines**: Greedy, EcoGreedy, and Threshold heuristics

Our key contributions include: (1) a multi-objective reward formulation balancing cost, emissions, and battery wear; (2) systematic hyperparameter ablation for both DQN and SARSA; (3) statistical evaluation with confidence intervals; and (4) the finding that SARSA with tile coding outperforms DQN in this domain.

2 Methods

2.1 Environment Formulation

We model the smart grid battery management problem as an MDP with the following components:

State Space $\mathcal{S} \subset \mathbb{R}^5$: The state vector consists of:

$$s_t = [\text{hour}_t, \text{SoC}_t, P_{\text{net},t}, c_{\text{grid},t}, I_{\text{CO}_2,t}] \tag{1}$$

where $\text{hour}_t \in [0, 24]$ is the time of day, $\text{SoC}_t \in [0, 1]$ is the battery state of charge, $P_{\text{net},t}$ is the net load (consumption minus solar), $c_{\text{grid},t}$ is the electricity price (/kWh), and $I_{\text{CO}_2,t}$ is the grid carbon intensity (gCO_2/kWh).

Action Space $\mathcal{A} = \{0, 1, 2\}$: The agent selects among three discrete actions: charge (0), discharge (1), or idle (2).

Reward Function: We define a multi-objective reward:

$$R_t = -(\alpha \cdot \text{Cost}_t + \beta \cdot \text{Emissions}_t + \gamma \cdot |a_t - a_{t-1}|) \tag{2}$$

where $\alpha = 1.0$, $\beta = 0.6$, and $\gamma = 0.05$ weight the electricity cost, carbon emissions, and action switching penalty respectively.

Battery Dynamics: We simulate a 10 kWh battery with 3 kW maximum power and 95% charge/discharge efficiency. Episodes span 24 hours with 30-minute time steps (48 steps per episode).

Synthetic Data Generation: The environment generates realistic daily profiles for each episode. Solar production follows a bell curve centered at midday with a randomly varying peak capacity (2.5–4.0 kW). Household consumption combines a 1.4 kW base load with Gaussian-shaped morning (7:30) and evening (19:00) demand peaks, plus random noise. Electricity prices follow time-of-use patterns with an evening premium (range 0.08–0.45 /kWh), and carbon intensity is higher during evening peak hours when fossil fuel plants ramp up (range 150–700 gCO_2/kWh). Random variation in all profiles ensures each episode represents a unique day.

2.2 Deep Q-Network (DQN)

DQN approximates the action-value function using a neural network $Q(s, a; \theta)$. We employ experience replay and a target network for stability [1]. The loss function is:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right] \tag{3}$$

We conduct an ablation study over three network architectures: Small [32, 32], Medium [64, 64], and Large [128, 64] hidden layers.

2.3 SARSA with Tile Coding

SARSA is an on-policy TD control algorithm with update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right] \tag{4}$$

For function approximation, we use tile coding [2], which creates sparse binary feature vectors through multiple overlapping tilings of the state space. Each tiling partitions the continuous state into discrete tiles, and the value function is a linear combination:

$$Q(s, a) = \mathbf{w}_a^\top \phi(s) \tag{5}$$

where $\phi(s)$ is the sparse tile coding feature vector. Based on insights from prior work, we use 32 tilings with 4 tiles per dimension, providing fine resolution with minimal aliasing.

2.4 Baseline Agents

We compare against four baselines:

- **Greedy**: Charges when price < 0.15 /kWh, discharges when > 0.28 /kWh
- **EcoGreedy**: Charges when carbon $< 280\text{gCO}_2$/kWh, discharges when> 420
- **Threshold**: Time-based schedule (charge hours 0–6, discharge hours 17–21)
- **Random**: Uniform random action selection (lower bound)
- **Idle**: No battery usage (reference for savings calculation)

3 Experimental Setup

Training: DQN agents are trained for 2,000 episodes per configuration with ϵ-greedy exploration (ϵ: $1.0 \rightarrow 0.05$, decay 0.995). SARSA agents are trained for 3,000 episodes with slower decay (ϵ: $1.0 \rightarrow 0.02$, decay 0.998) to account for on-policy sample inefficiency.

Evaluation: All agents are evaluated over 100 episodes with fixed random seeds. We report mean performance with 95% confidence intervals computed using the t-distribution.

Metrics: We measure (1) cumulative reward, (2) daily electricity savings vs. idle baseline (), and (3) CO_2 emissions avoided vs. baseline (kg).

Implementation: Code follows the RL-GLUE framework [3] with modular agent and environment interfaces. DQN uses PyTorch; SARSA requires only NumPy.

Table 1. DQN architecture ablation study (2,000 training episodes).

Configuration	Architecture	Mean Reward	Training Time
DQN-Small	[32, 32]	−13.68	532s
DQN-Medium	[64, 64]	**−13.26**	510s
DQN-Large	[128, 64]	−14.61	518s

Table 2. SARSA variant comparison (3,000 training episodes, 32 tilings × 4 tiles).

Variant	Mean Reward (last 100)	Training Time
SARSA	**−10.98**	75s
Expected SARSA	−11.24	79s

4 Results and Discussion

4.1 DQN Hyperparameter Ablation

Table 1 presents the DQN ablation results. The Medium architecture [64, 64] achieves the best performance, suggesting that larger networks may overfit to the relatively low-dimensional state space.

4.2 SARSA Variant Comparison

We compare standard SARSA against Expected SARSA, which uses the expected value under the policy rather than the sampled next action. As shown in Table 2, standard SARSA slightly outperforms Expected SARSA in our setting, achieving a final reward of −10.98.

4.3 Final Agent Comparison

Table 3 presents the comprehensive evaluation results with 95% confidence intervals. SARSA achieves the best overall performance across all metrics, with statistically significant improvements over DQN.

4.4 Discussion

SARSA outperforms DQN: Surprisingly, SARSA with tile coding achieves 15.9% better reward than the best DQN configuration. We attribute this to: (1) the structured nature of the state space, where tile coding provides appropriate inductive bias; (2) SARSA's training stability compared to DQN's potential for overestimation; and (3) more extensive exploration through slower ϵ decay.

Sample efficiency vs. asymptotic performance: While DQN's experience replay improves sample efficiency, SARSA's simpler updates lead to better final performance given sufficient training episodes.

Table 3. Agent comparison with 95% confidence intervals (100 evaluation episodes).

Agent	Type	Reward	Savings ()	CO_2 Avoided (kg)
SARSA	Learned	$\mathbf{-10.88 \pm 0.79}$	$\mathbf{2.194 \pm 0.267}$	$\mathbf{3.336 \pm 0.411}$
DQN	Learned	-12.93 ± 1.68	1.131 ± 0.708	1.888 ± 1.099
Threshold	Rule-based	-12.98 ± 0.72	1.274 ± 0.366	1.450 ± 0.627
EcoGreedy	Rule-based	-14.16 ± 0.88	0.760 ± 0.477	0.543 ± 0.759
Greedy	Rule-based	-14.48 ± 0.85	0.551 ± 0.480	0.448 ± 0.698
Idle	Baseline	-14.82 ± 0.51	0.000 ± 0.000	0.000 ± 0.000
Random	Baseline	-21.91 ± 1.88	-2.427 ± 0.892	-4.175 ± 1.478

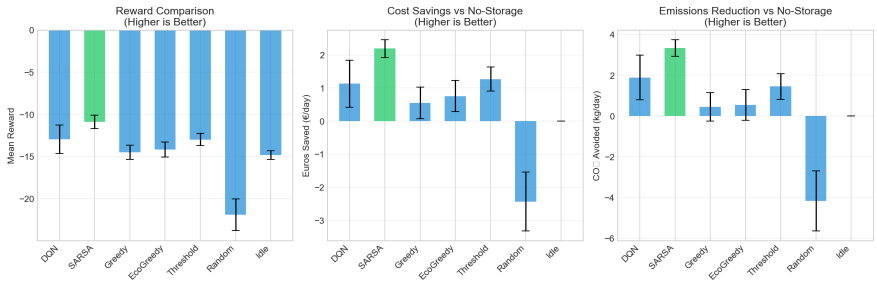

Fig. 1. Agent comparison with error bars: (Left) Mean reward, (Center) Daily savings, (Right) CO_2 avoided. Higher values are better for all metrics.

Threshold baseline performs well: The time-based Threshold agent achieves competitive performance, demonstrating that domain knowledge (charge at night, discharge in evening) captures significant value. However, learned agents generalize better to varying price patterns.

5 Conclusion

We presented a comprehensive study of reinforcement learning for smart grid battery management. Our multi-objective formulation balances electricity cost, carbon emissions, and battery wear. Through systematic evaluation, we demonstrated that:

1. SARSA with tile coding achieves the best performance (2.19 daily savings, 3.34 kg CO_2 avoided), outperforming DQN by 15.9% in mean reward.
2. Medium-sized DQN architectures [64, 64] outperform both smaller and larger networks.
3. Linear function approximation with appropriate features can compete with deep learning in structured domains.
4. The Threshold heuristic provides a strong baseline, highlighting the value of domain knowledge.

A Learning Curves and Visualizations

Fig. 2. Comprehensive learning curves for DQN architectures and SARSA variants.

References

1. Mnih, V., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)
2. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. 2nd edn. MIT Press, Cambridge (2018)
3. Tanner, B., White, A.: RL-Glue: Language-independent software for reinforcement-learning experiments. Journal of Machine Learning Research **10**, 2133–2136 (2009)
4. Vázquez-Canteli, J.R., Nagy, Z.: Reinforcement learning for demand response: A review of algorithms and modeling techniques. Applied Energy **235**, 1072–1089 (2019)
5. Rocchetta, R., Bellani, L., Compare, M., Zio, E., Patelli, E.: A reinforcement learning framework for optimal operation of a battery in presence of renewable energy sources. Renewable Energy **139**, 504–515 (2019)